Iván N. Guzmán

Programación Logica y Funcional | 3 | Nov | 26

2.3 Intervalos

Un intervalo (range) es una notación que permite construir listas a partir de secuencias aritmeticas, sin escribir cada elemento de forma manual. En Haskell, la expresión [1..10] genera la lista de los enteros del 1 al 10, mientras que [2,4..10] indica un paso definido por los dos primeros elementos y produce [2,4,6,8,10] (Lipovača, 2011). Según el Haskell 2010 Language Report, estas expresiones son azúcar sintáctica de las funciones enumFrom, enumFromThen, enumFromTo y enumFromThenTo, que pertenece ala clase de tipo Enum (Marlow, 2010). Por ello, los intervalos no se limitan a números enteros, sino que funcionan con cualquier tipo que se instancia de Enum, como los caracteres ['a'..'e'] produce "abcde".

Los intervalos también admiten pasos descendentes, como [10,8..0], y pueden ser infinitos si se omite el ~~numero~~ límite superior, como en [1..]. Esto es solo práctico gracias a la evaluación perezosa, pues la lista se construye conforme se consume; por ejemplo, take 5 [1..] devuelve [1,2,3,4,5] (Hutton, 2016). Con números decimales conviene tener cuidado, ya que el reporte del lenguaje indica que la secuencia puede incluir elementos ligeramente mayores al límite superior (Marlow, 2010).

2.4 Operadores

En Haskell, un operador es una función cuyo nombre está formado por símbolos y que se escribe por defecto en forma infija, por lo que 2+3 equivale (+) 2 3 (Marlow, 2010). Del mismo modo cualquier función de dos argumentos puede usarse como operador si se encierra entre acentos graves, como en 10 `div` 3 (Lipovača, 2011).

Cada operador tiene una precedencia (de 0 a 9) y una asociatividad (izquierda, derecha o ninguna), que se definen como declaraciones de fijeza (infixl, infixr, e infix). Si el programador no declara la fijeza de un operador propio, se

asume infixl 9 (Marlow, 2010). Por ejemplo, * tiene mayor precedencia (7) que + (6), por lo que 2 + 3 * 4 da 14.

También existen las secciones, que son la aplicación parcial de un operador infijo: (+1) suma uno a su argumento y (2*) lo multiplica por dos (Hutton, 2026). Finalmente, el operador . compone funciones, de modo que (f . g) x equivale a f (g x), y $ evita paréntesis por tener la precedencia más baja (Lipovača, 2011). Hughes (1989) sostiene que combinar funciones pequeñas con este tipo de operadores es la base de la modularidad en la programación funcional.

2.7 Evaluación perezosa.

La evaluación perezosa es una estrategia en la que una expresión no se evalúa hasta que su valor es necesario y, una vez evaluada, el resultado se comparte para no repetir el cálculo (Hutton, 2016). En Haskell esto es habitual, y Hudak et al. (2007) señalan que fue el principal motivo que llevó a mantener el lenguaje puro.

Sus ventajas principales son tres. Primero, evita cálculos innecesarios: fst (1, undefined) devuelve 1 sin error. Segundo, permite estructuras infinitas, como fibs = 0 : 1 : zipWith (+) fibs (tail fibs) de las que se consume solo lo necesario con take 10 fibs (Hutton, 2016). Tercero, mejora la modularidad, porque separa el generador de datos del consumidor que decide cuántos necesita (Hughes, 1989).

Como desventaja, el sistema debe guardar expresiones pendientes (thunks), lo que puede causar un alto consumo de memoria, conocido como fugas de espacio. Para controlarlo, Haskell ofrece la función seq y el operador $!, que fuerzan la evaluación de un argumento (Hutton, 2016).

Referencias

Hudak, P., Hughes, J., Peyton Jones, S., & Wadler, P. (2007). A

Iván N. Guzmán

Programación Lógica y Funcional

history of Haskell: Being lazy with class. En Proceedings of the third ACM SIGPLAN conference on History of programming languages (pp. 12-1-12-55). ACM https://doi.org/10.1145/1238844.1238856

Hughes, J. (1989). Why functional programming matters. The Computer Journal, 32(2), 98-107. https://doi.org/10.1093/comjnl/32.2.98

Hutton, G. (2016). Programming in Haskell (2ª ed.) Cambridge University Press

Lipovača, M. (2011). Learn you a Haskell for great good! A beginners guide. No starch Press. https://learnyouahaskell.github.io/

Marlow, S. (Ed). (2010). Haskell 2010 language report. Haskel.org https://www.haskell.org/onlinereport/haskell2010/